

APPLICAZIONE MATCHUP

CORSO DI INGEGNERIA DEL SOFTWARE A.A. 2025 / 2026

DOCUMENTO DI PROGETTO ITERAZIONE 4

*SALVATORE CAMPIONE
MARIO CIPRIANO*

Sommario

1.REQUISITI	3
2.ANALISI.....	3
2.1 MODELLO DI DOMINIO	3
2.2 DIAGRAMMA DI SEQUENZA DI SISTEMA DI UC9	4
2.3 DIAGRAMMA DI SEQUENZA DI SISTEMA DI UC10	6
2.4 DIAGRAMMA DI SEQUENZA DI SISTEMA DI UC11	7
2.5 CONTRATTI DELLE OPERAZIONI DI UC9	7
2.5.1 ANNULLA PRENOTAZIONE.....	7
2.5.2 CONFERMA ANNULLAMENTO PRENOTAZIONE.....	8
2.6 CONTRATTI DELLE OPERAZIONI DI UC10	8
2.6.1 RICARICA CONTO	8
2.6.2 CONFERMA RICARICA	8
2.7 CONTRATTI DELLE OPERAZIONI DI UC11	9
2.7.1 RICARICA CONTO GIOCATORE.....	9
2.7.2 CONFERMA RICARICA CONTO GIOCATORE	9
3.PROGETTAZIONE.....	9
3.1 DIAGRAMMI DI SEQUENZA DI UC9	10
3.1.1 MOSTRA PRENOTAZIONI IN CORSO	10
3.1.2 ANNULLA PRENOTAZIONE.....	10
3.1.3 CALCOLA RIMBORSO	11
3.1.4 CONFERMA ANNULLAMENTO PRENOTAZIONE.....	11
3.1.5 EFFETTUA RIMBORSO	11
3.2 DIAGRAMMI DI SEQUENZA DI UC10	11
3.2.1 RICARICA CONTO	12
3.2.2 CONFERMA RICARICA	12
3.3 DIAGRAMMI DI SEQUENZA DI UC11	12
3.3.1 RICARICA CONTO GIOCATORE.....	12
3.3.2 CONFERMA RICARICA CONTO GIOCATORE	13
3.4 AGGIORNAMENTO DIAGRAMMA DI SEQUENZA DI EFFETTUA ISCRIZIONE TORNEO	13
3.4.1 EFFETTUA ISCRIZIONE TORNEO	13
3.5 DIAGRAMMA DELLE CLASSI DI PROGETTO	14
4.PATTERN UTILIZZATI	14
5. FASE DI TESTING.....	15
5.1 INTRODUZIONE AL TESTING	15
5.2 SCENARIO DI ANNULLAMENTO PRENOTAZIONE.....	16
5.3 SCENARIO RICARICA CONTO	16
5.4 SCENARIO DI RICARICA CONTO GIOCATORE	17

1.REQUISITI

In questa quarta iterazione andremo ad implementare il caso d'uso 10 e 11 che rappresentano rispettivamente le funzionalità di ricarica di cui possono usufruire i giocatori, la ricarica autonoma del proprio saldo residuo, oppure la ricarica tramite l'intervento del gestore della struttura. Inoltre, sempre in questa iterazione andremo a gestire anche l'eventuale possibilità del giocatore di poter annullare le proprie prenotazioni e di conseguenza permettere il rimborso della quota pagata per esse, per il quale sono state individuate e seguite le apposite regole di dominio. Quindi ricapitolando in questa iterazione andremo ad eseguire:

- Implementazione dell'UC 9 (Annullamento prenotazione);
- Implementazione dell'UC 10(Effettua ricarica conto);
- Implementazione dell'UC11(effettua ricarica conto giocatore).

2.ANALISI

Come nelle iterazioni precedenti ,anche qui andiamo a riportare il diagramma del modello di dominio, i diagrammi di sequenza di sistema e i contratti delle operazioni.

2.1 MODELLO DI DOMINIO

In questa iterazione il modello di dominio rimane identico a quello dell'iterazione tre in quanto non abbiamo introdotto nessuna nuova entità.

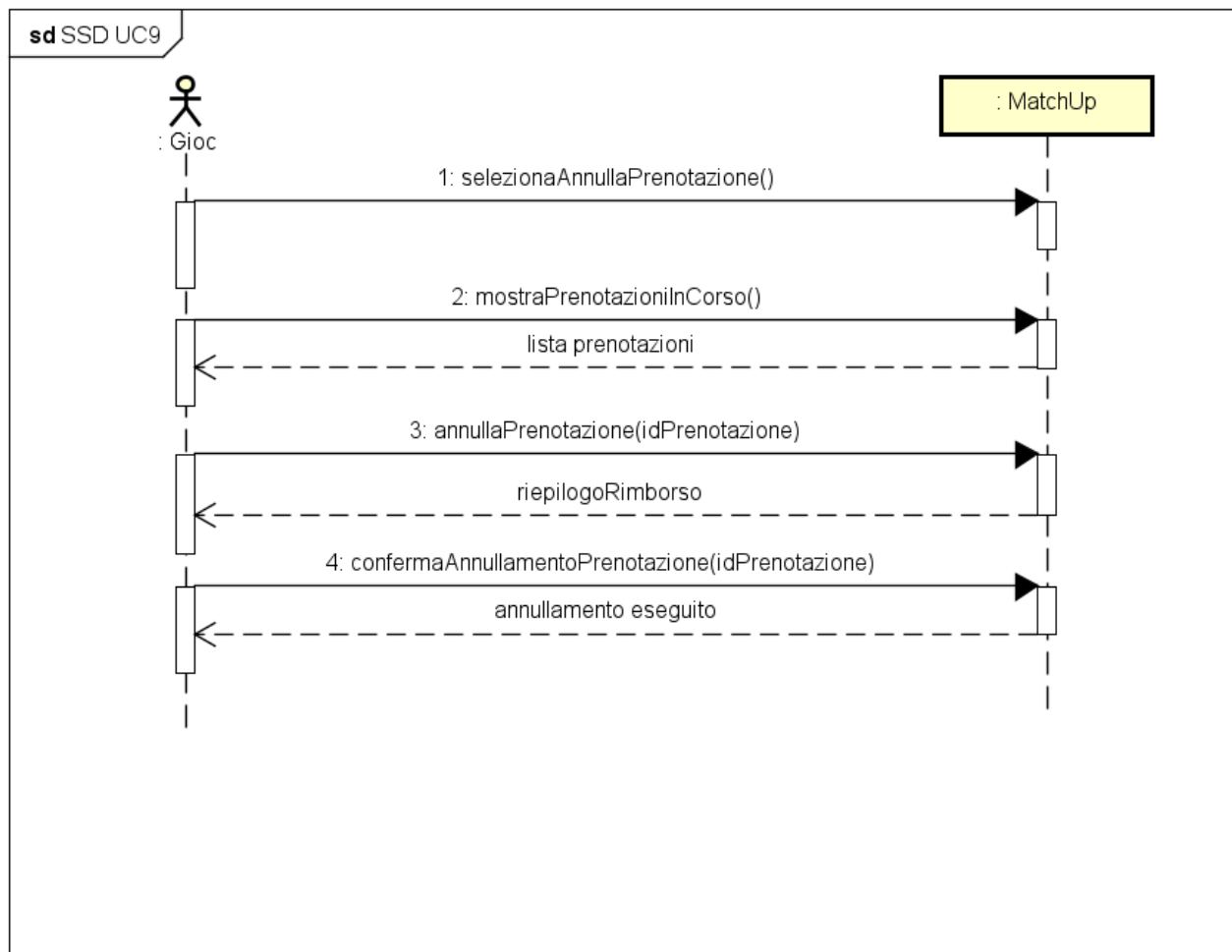


Figura 2: Diagramma di sequenza di sistema di UC9

2.3 DIAGRAMMA DI SEQUENZA DI SISTEMA DI UC10

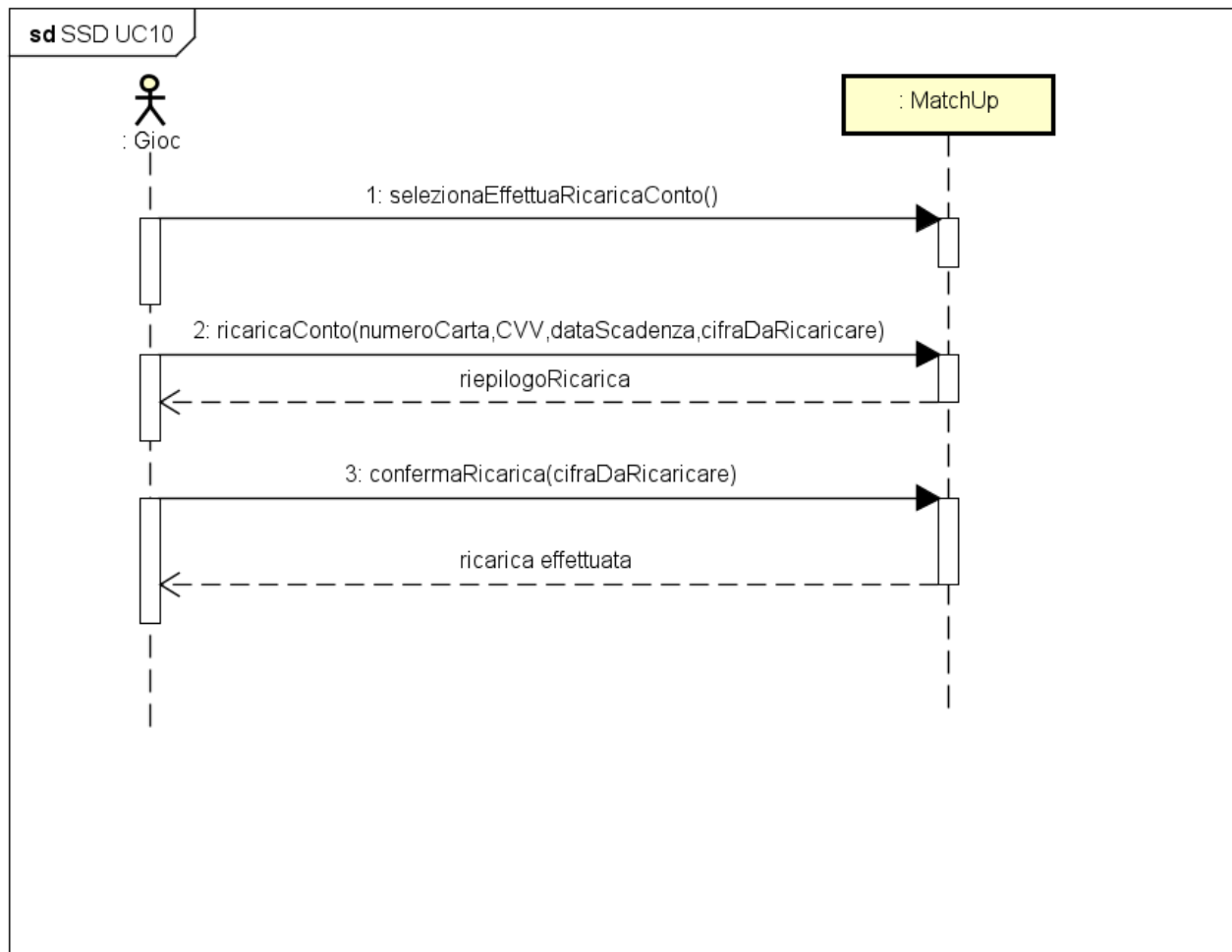


Figura 3: Diagramma di sequenza di sistema di UC10

2.4 DIAGRAMMA DI SEQUENZA DI SISTEMA DI UC11

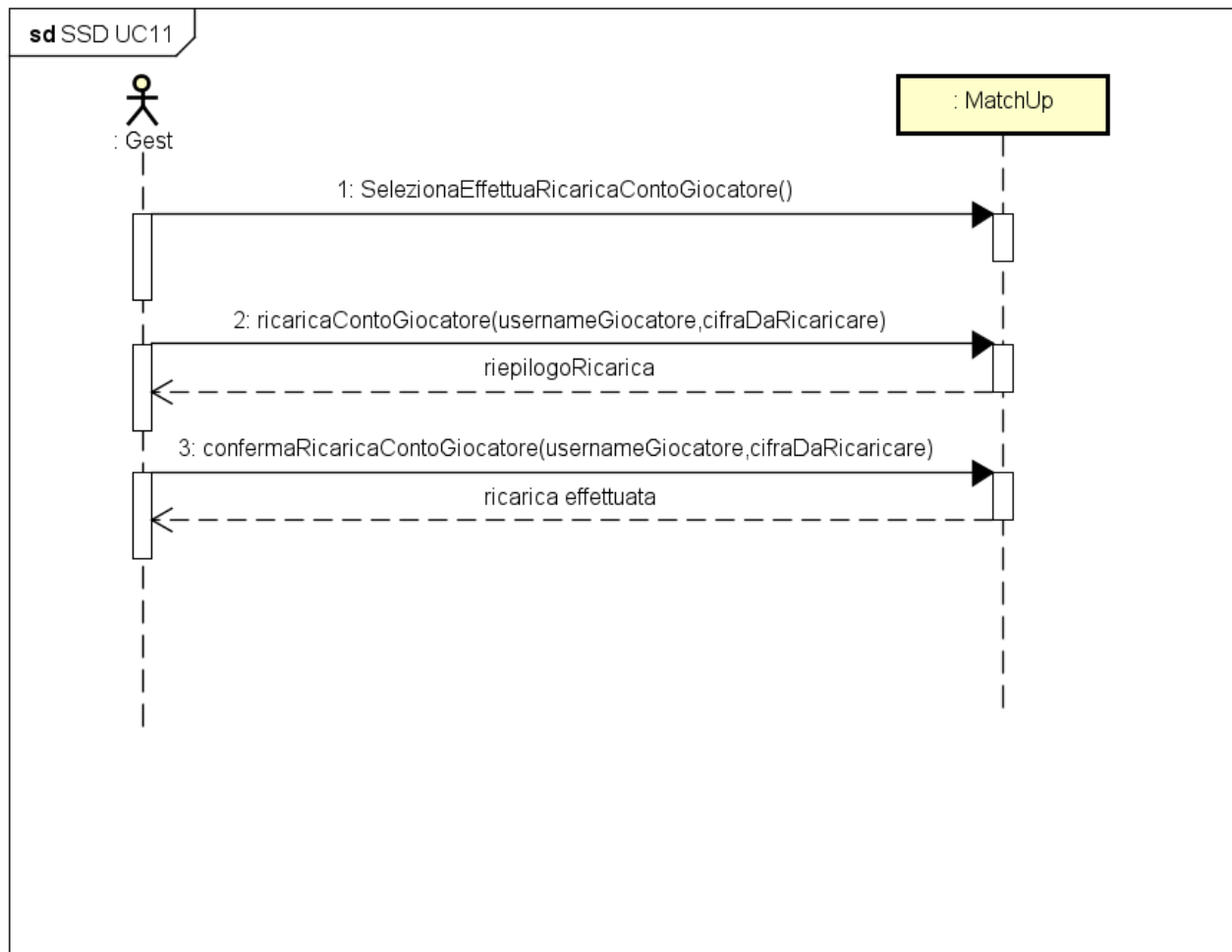


Figura 4: Diagramma di sequenza di sistema di UC11

2.5 CONTRATTI DELLE OPERAZIONI DI UC9

2.5.1 ANNULLA PRENOTAZIONE

Questa operazione permette al giocatore di poter annullare le prenotazioni che ha effettuato in precedenza.

Operazione	AnnullaPrenotazione(idPrenotazione)
Riferimenti	Caso d'uso UC9 (Annullamento prenotazione)
Pre-condizioni	- Il giocatore g è loggato su MatchUp - Esiste un'istanza p di prenotazione associata al giocatore g
Post-condizioni	È stato calcolato l'importo del rimborso in base alle politiche di cancellazione del sistema e memorizzato temporaneamente per la conferma.

Tabella 1: Contratto dell'operazione Annulla prenotazione

2.5.2 CONFERMA ANNULLAMENTO PRENOTAZIONE

Questa operazione conferma l'annullamento della prenotazione del giocatore e permette il rimborso sul suo conto personale

Operazione	ConfermaAnnullamentoPrenotazione(idPrenotazione)
Riferimenti	Caso d'uso UC9 (Annullamento prenotazione)
Pre-condizioni	- Il giocatore g è loggato su MatchUp. - Esiste un'istanza p di Prenotazione associata al giocatore g
Post-condizioni	- L'attributo stato della prenotazione p è stata settato su "rimborsata" - L'attributo saldo del giocatore g viene incrementato del costo della prenotazione (rimborso) - E' stata settata libera la fascia oraria occupata dalla prenotazione del giocatore.

Tabella 2: Contratto dell'operazione Conferma annullamento prenotazione

2.6 CONTRATTI DELLE OPERAZIONI DI UC10

2.6.1 RICARICA CONTO

Questa operazione permette al giocatore di ricaricare il proprio conto personale

Operazione	RicaricaConto(numeroCarta:String,CCV:Int,dataScadenza:GregorianCalendar, cifraDaRicaricare:double)
Riferimenti	Caso d'uso UC10 (Effettua ricarica conto)
Pre-condizioni	-Il giocatore g è loggato su MatchUp
Post-condizioni	Nessuna

Tabella 3: Contratto dell'operazione Ricarica conto

2.6.2 CONFERMA RICARICA

Questa operazione permette di conferma l'operazione di ricarica del conto

Operazione	ConfermaRicarica(numeroCarta:String,CCV:int,dataScadenza:GregorianCalendar,cifraDaRicaricare:double)
Riferimenti	Caso d'uso UC10 (Effettua ricarica conto)
Pre-condizioni	-Il giocatore g è loggato su MatchUp
Post-condizioni	- E' stato incrementato il valore dell'attributo saldo del giocatore g della quantità "cifraDaRicaricare"

Tabella 4: Contratto dell'operazione Conferma ricarica conto

2.7 CONTRATTI DELLE OPERAZIONI DI UC11

2.7.1 RICARICA CONTO GIOCATORE

Questa operazione permette al gestore di accreditare sul conto del giocatore la cifra da egli richiesta.

Operazione	RicaricaContoGiocatore(username:String,cifraDaRicaricare:double)
Riferimenti	Caso d'uso UC11 (Effettua ricarica conto giocatore)
Pre-condizioni	-Il gestore ge è loggato su MatchUp -Il giocatore g è registrato su MatchUp
Post-condizioni	Nessuna

Tabella 5: Contratto dell'operazione Ricarica conto giocatore

2.7.2 CONFERMA RICARICA CONTO GIOCATORE

Questa operazione permette al gestore di confermare l'operazione di ricarica effettuata per conto del giocatore

Operazione	ConfermaRicaricaContoGiocatore(username:String,cifraDaRicaricare:double)
Riferimenti	Caso d'uso UC11 (Effettua ricarica conto giocatore)
Pre-condizioni	-Il gestore ge è loggato su MatchUp -Il giocatore g è registrato su MatchUp
Post-condizioni	- E' stato incrementato il valore dell'attributo saldo del giocatore g della quantità " cifraDaRicaricare"

Tabella 6: Contratto dell'operazione Conferma ricarica conto giocatore

3.PROGETTAZIONE

Come per le iterazioni precedenti anche qui andremo a realizzare e mostrare i diagrammi di sequenza e il diagramma delle classi di progetto.

3.1 DIAGRAMMI DI SEQUENZA DI UC9

3.1.1 MOSTRA PRENOTAZIONI IN CORSO

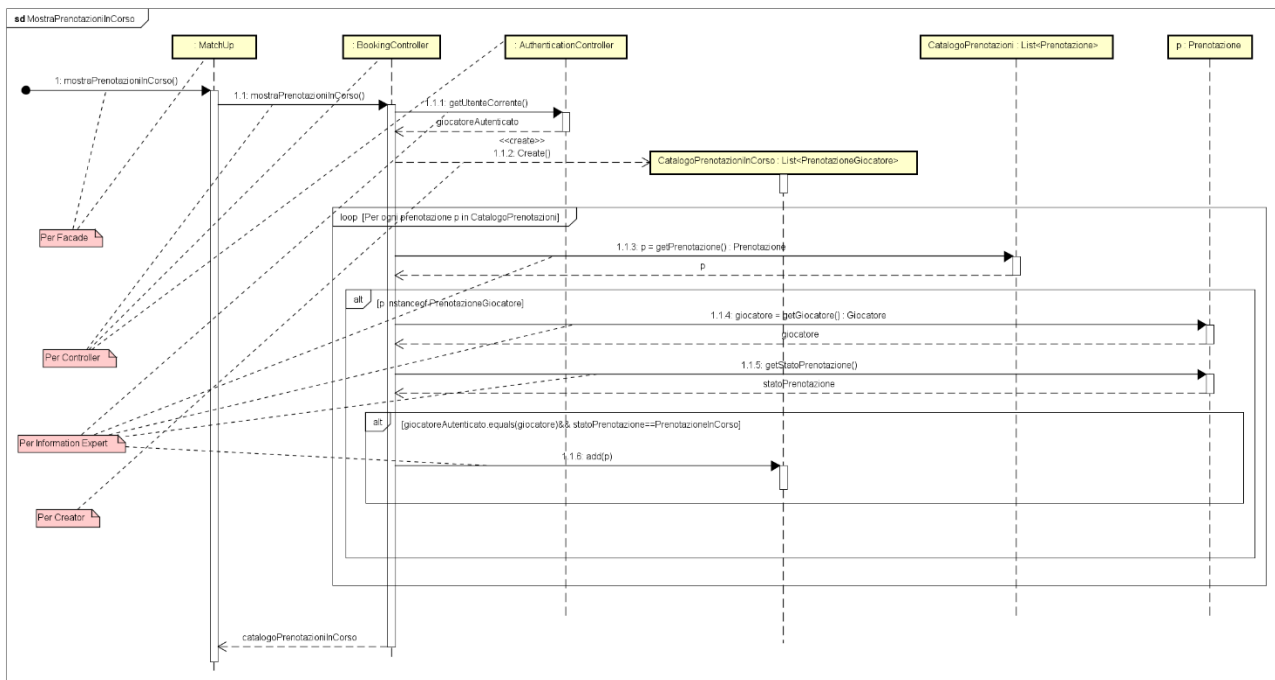


Figura 5: Diagramma di sequenza di MostraPrenotazioniInCorso

3.1.2 ANNULLA PRENOTAZIONE

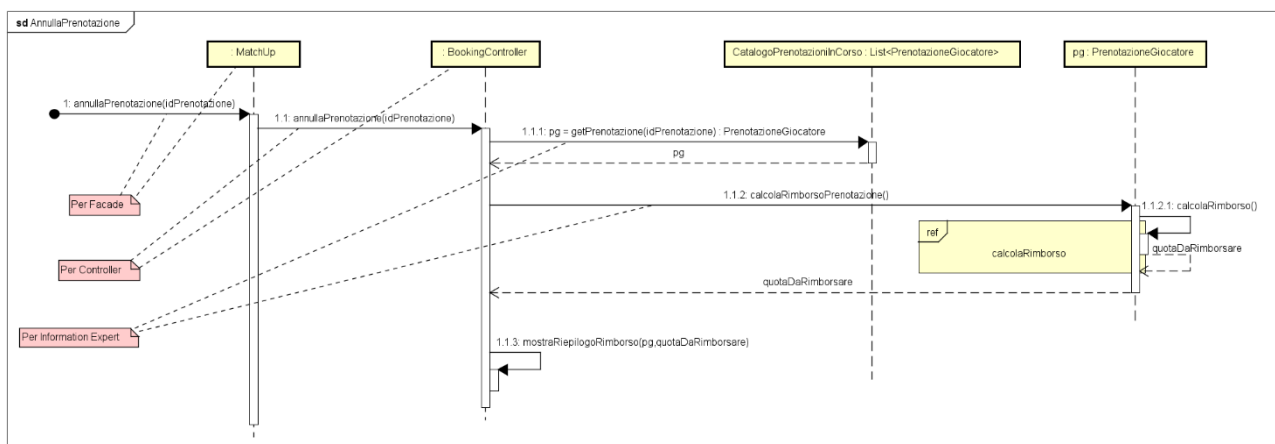


Figura 6: Diagramma di sequenza di AnnullaPrenotazione

3.1.3 CALCOLA RIMBORSO

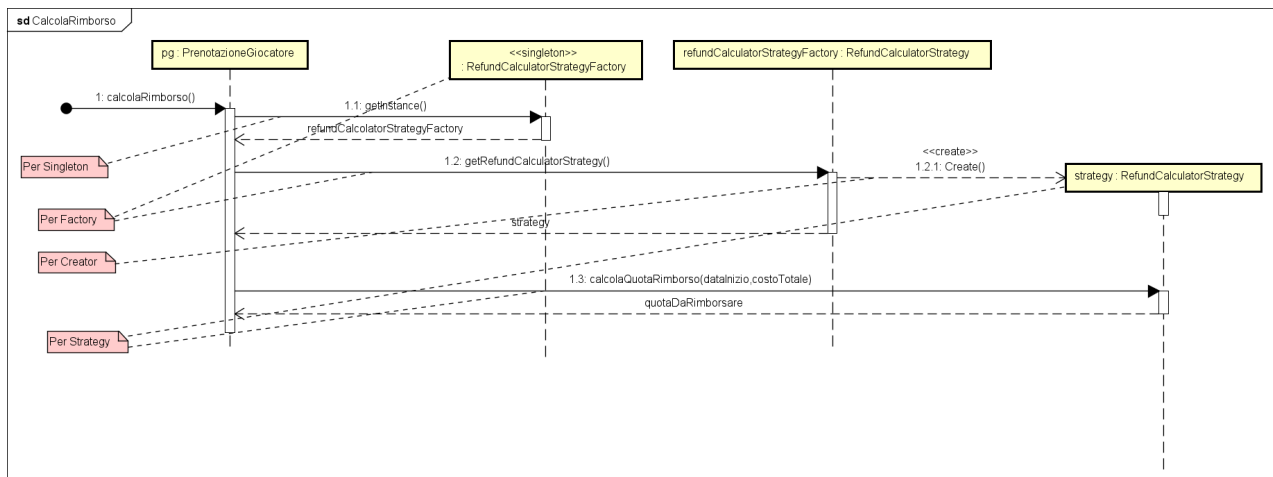


Figura 7: Diagramma di sequenza di CalcolaRimborso

3.1.4 CONFERMA ANNULLAMENTO PRENOTAZIONE

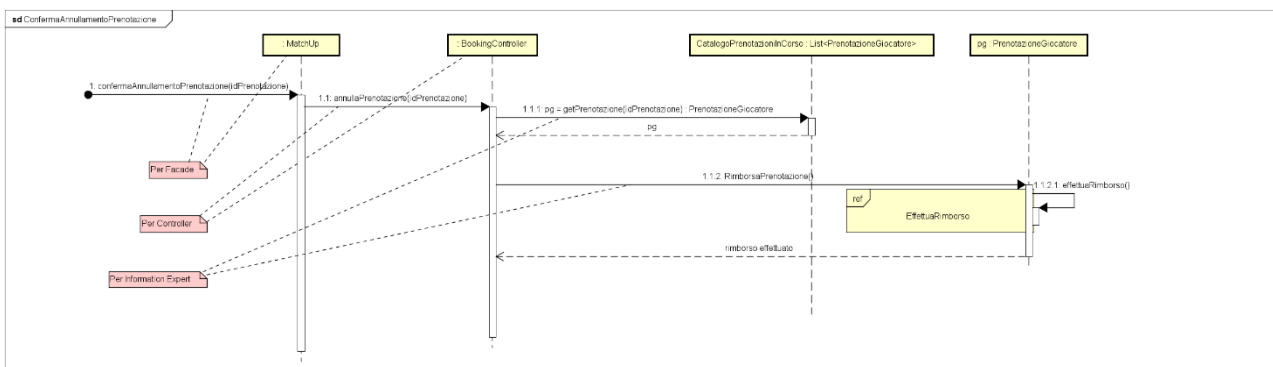


Figura 8: Diagramma di sequenza di ConfermaAnnullamentoPrenotazione

3.1.5 EFFETTUA RIMBORSO

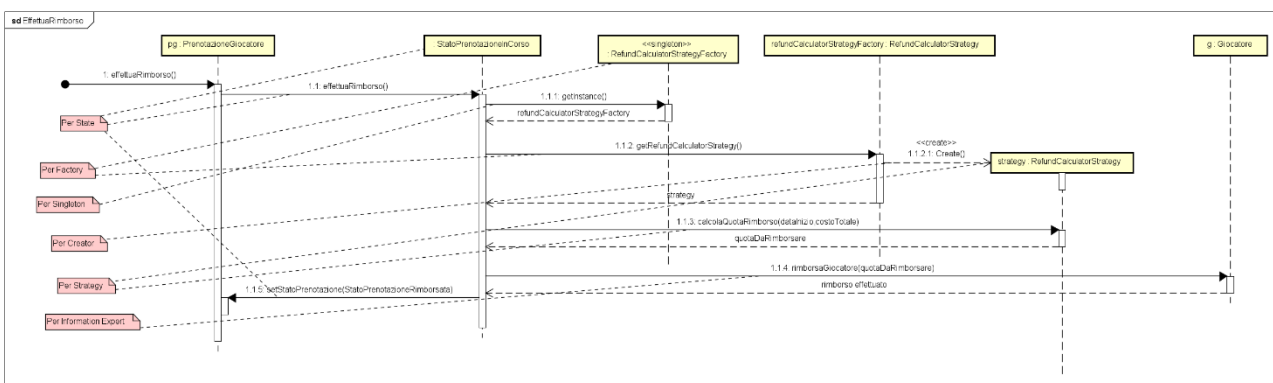


Figura 9: Diagramma di sequenza di EffettuaRimborso

3.2 DIAGRAMMI DI SEQUENZA DI UC10

Per la gestione delle operazioni di ricarica nei casi UC10 e UC11 è stato introdotto un nuovo controller di caso d'uso chiamato WalletController, che verrà utilizzato insieme agli altri già definiti nell'iterazione precedente

3.2.1 RICARICA CONTO

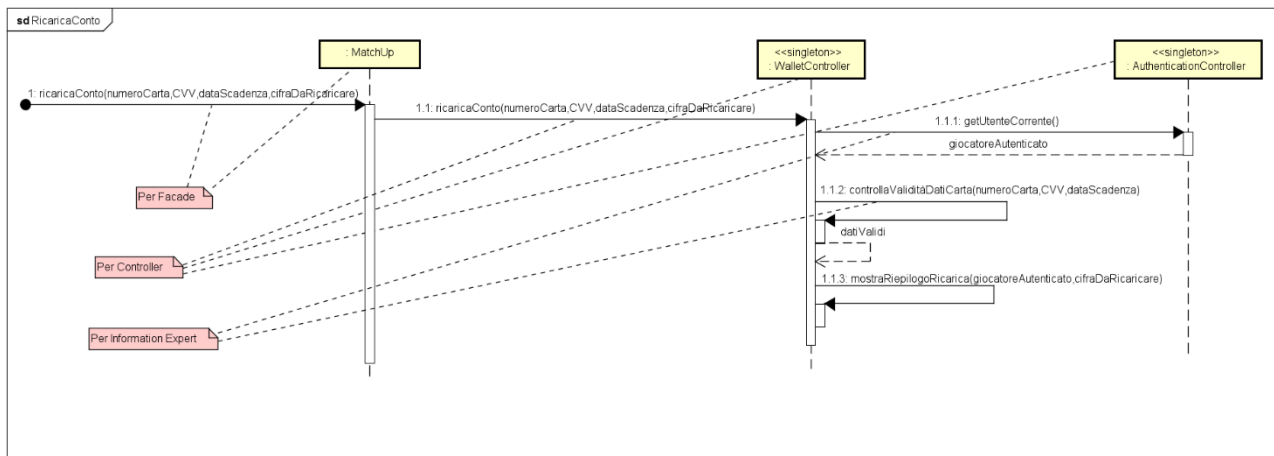


Figura 10: Diagramma di sequenza di RicaricaConto

3.2.2 CONFERMA RICARICA

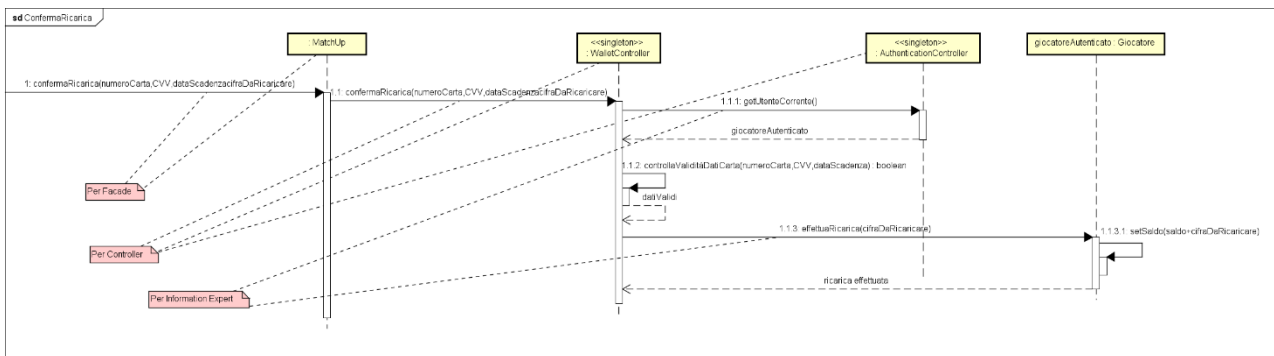


Figura 11: Diagramma di sequenza di ConfermaRicarica

3.3 DIAGRAMMI DI SEQUENZA DI UC11

3.3.1 RICARICA CONTO GIOCATORE

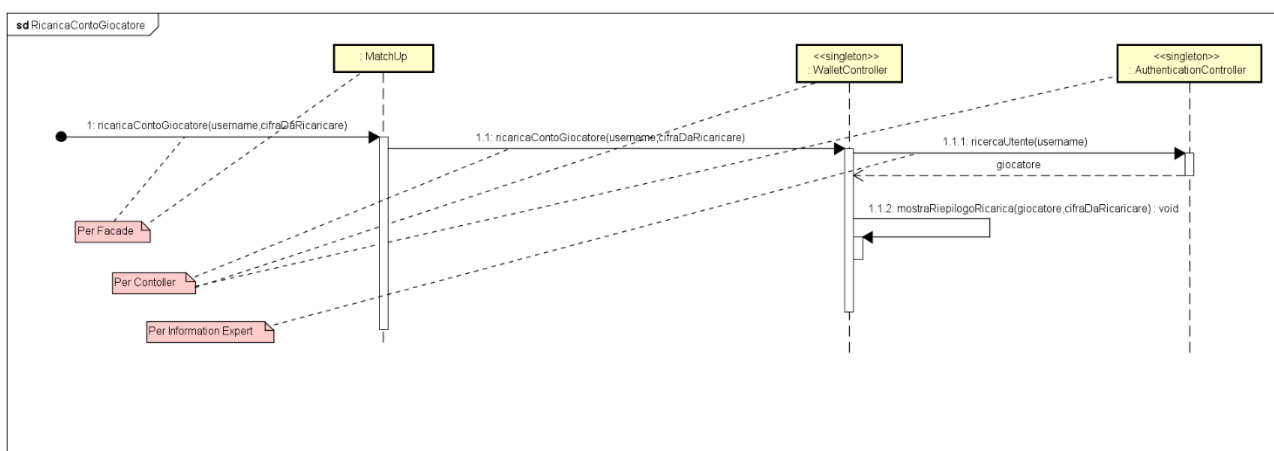


Figura 12: Diagramma di sequenza di RicaricaContoGiocatore

3.3.2 CONFERMA RICARICA CONTO GIOCATORE

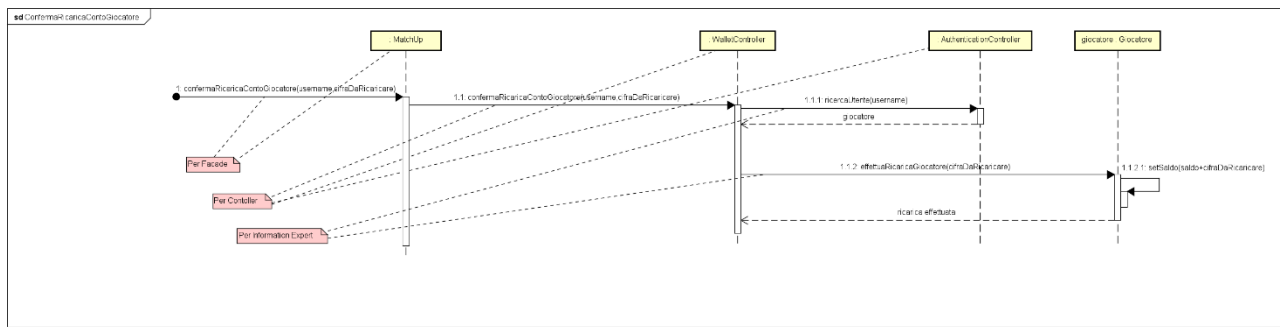


Figura 13: Diagramma di sequenza di ConfermaRicaricaContoGiocatore

3.4 AGGIORNAMENTO DIAGRAMMA DI SEQUENZA DI EFFETTUA ISCRIZIONE TORNEO

In questa iterazione abbiamo deciso di modificare lo scalamento del saldo facendo in modo che a tutti i membri della squadra venisse scalata la quota di iscrizione al torneo.

3.4.1 EFFETTUA ISCRIZIONE TORNEO

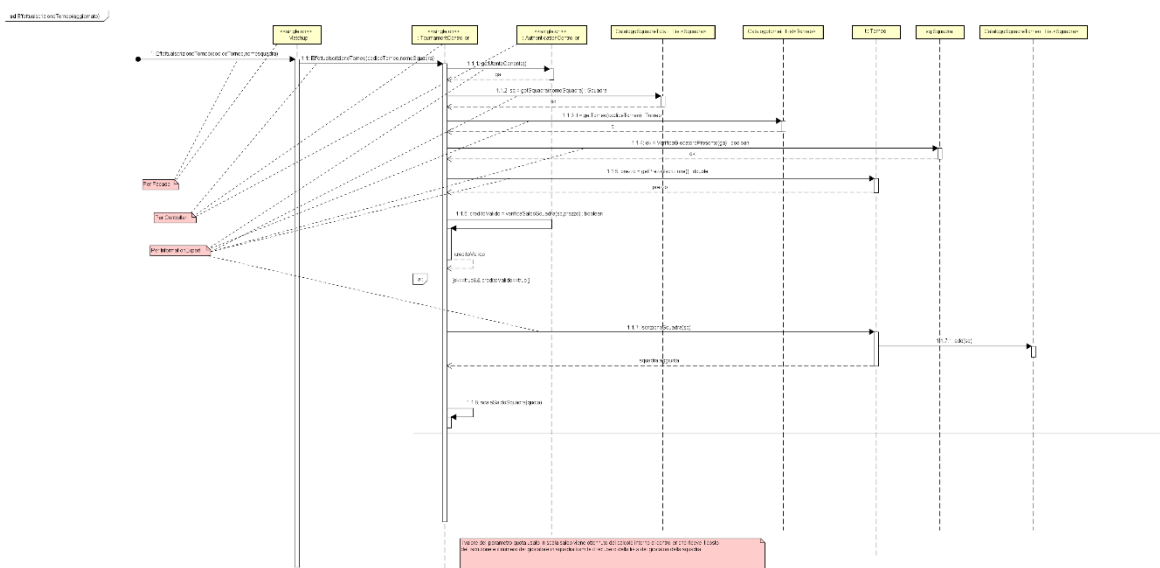


Figura 14: Diagramma di sequenza di Effettua iscrizione torneo

4.PATTERN UTILIZZATI

- **Pattern Strategy:** Applicato per la gestione del calcolo dei rimborsi (RefundCalculatorStrategy). L'utilizzo di un'interfaccia comune e di classi di strategie concrete permette di incapsulare le diverse politiche di rimborso (es. rimborso totale, parziale o nullo in base alle scadenze), rendendo il sistema estendibile qualora si volessero introdurre nuove regole di calcolo senza modificare il codice esistente.
- **Pattern Factory:** Utilizzato in sinergia con il pattern Strategy tramite la classe

RefundCalculatorStrategyFactory. Questo pattern si occupa di delegare e centralizzare la logica di creazione delle istanze concrete di strategia di rimborso, disaccoppiando il client dalla loro istanziazione.

- **Pattern Singleton:** Applicato su componenti chiave del sistema, come i controller globali e le factory per garantire che esista una sola istanza di tali oggetti durante tutto il ciclo di vita dell'applicazione ed evitare la duplicazione dello stato.
- **Pattern State:** Applicato sulla classe *PrenotazioneGiocatore*. Quest'ultimo è un oggetto dinamico in quanto durante il suo ciclo di vita può assumere stati differenti:
- **StatoPrenotazioneInCorso:** viene impostato questo stato quando l'oggetto viene istanziato e permane finché la data di fine prenotazione non viene raggiunta;
- **StatoPrenotazioneConclusa:** viene impostato quando la data di fine prenotazione risulta trascorsa;
- **StatoPrenotazioneRimborsata:** viene impostato quando l'utente ha confermato l'annullamento della propria prenotazione.

Gli stati sopra elencati prevedono l'implementazione del metodo *effettuaRimborso()* dove nei primi due casi consiste di una semplice stampa a video, mentre nell'ultimo si avrà l'accredito del rimborso previsto sul conto del giocatore e la modifica della fascia oraria del suddetto campo.

- **Pattern Controller:** L'utilizzo di controller dedicati per i casi d'uso (WalletController), insieme a quelli già visti nell'iterazione precedente funge da punto d'accesso coordinato per le richieste provenienti da MatchUp, separando la logica di presentazione dalla logica di business e di dominio.
- **Pattern Facade:** Come per l'iterazione precedente, anche qui applichiamo questo pattern alla classe MatchUp.

5. FASE DI TESTING

5.1 INTRODUZIONE AL TESTING

In questa iterazione si sono sviluppati una serie di casi d'uso orientati alla gestione dei di funzionalità per i giocatori: gli scenari di annullamento di prenotazioni e di ricarica dei conti personali prevedono diversi scenari di test, destinati a verificare la corretta applicazione delle regole di dominio introdotte.

5.2 SCENARIO DI ANNULLAMENTO PRENOTAZIONE

Lo scenario annullamento di prenotazione prevede la seguente sequenza di passaggi:

1. Ricerca della prenotazione da annullare;
2. Conferma annullamento prenotazione;
3. Emissione del rimborso della prenotazione (se previsto);
4. Liberazione della fascia oraria;

Dopo aver recuperato in ingresso dall'utente il codice della prenotazione da annullare, viene chiamato il metodo *annullaPrenotazione()* implementato nella classe *BookingController*. Se la prenotazione viene trovata, viene calcolato il rimborso in accordo alle regole di dominio e mostrato il riepilogo al giocatore.

Se l'annullamento viene confermato, tramite la chiamata al metodo *effettuarimborso()*, implementato nello *StatoPrenotazione*, viene emesso il riaccredito del rimborso al giocatore e liberata la fascia oraria. Affinché il rimborso avvenga correttamente, viene verificato che lo stato della prenotazione deve essere "in corso e non a conclusa oppure già rimborsata". Gli scenari di test individuati sono:

- a) Annullamento di una prenotazione esistente ed in corso. È lo scenario di successo.
- b) Annullamento di una prenotazione non esistente. È uno scenario di fallimento.
- c) Annullamento di una prenotazione esistente ma con stato non "in corso". È uno scenario di fallimento.

5.3 SCENARIO RICARICA CONTO

Lo scenario di ricarica conto permette al giocatore attualmente autenticato al servizio di ricaricare in autonomia il proprio conto. I passaggi da eseguire sono i seguenti:

1. Richiesta da parte dell'utente dei seguenti dati: cifra da ricaricare, numero di carta, data scadenza carta, CVV.
2. Verifica correttezza dati inseriti e generazione del riepilogo ricarica;
3. Conferma della ricarica.

Dopo aver eseguito il passaggio 1, viene chiamato il metodo *ricaricaConto()* implementato nella classe *WalletController*: tale metodo, dopo aver verificato i dati da parte del metodo *controllaValiditàDatiCarta()* implementato nella medesima classe, emette il riepilogo della ricarica carta. Il passaggio 3 richiede la conferma al giocatore del pagamento ed in caso di esito positivo si ha un accredito dell'importo sul conto del giocatore. Gli scenari di test individuati sono:

- a) Il numero di carta inserito non è appartenente ai seguenti circuiti: Mastercard, Visa, American Express. È uno scenario di fallimento.
- b) Il CVV non ha il formato corretto. È uno scenario di fallimento.
- c) La data di scadenza della carta è stata superata. È uno scenario di fallimento.
- d) L'importo inserito è superiore a 300 € o inferiore a 10 €.
- e) I dati della carta sono corretti e la ricarica viene effettuata. È lo scenario di successo.

5.4 SCENARIO DI RICARICA CONTO GIOCATORE

L'ultimo scenario di testing riguarda la ricarica da parte dei gestori della struttura, dei conti dei vari giocatori. Questo scenario è un'alternativa del precedente, in quanto nel caso un giocatore non possieda una carta di credito egli può comunque usare il servizio. Il gestore, a richiesta del giocatore, si fa comunicare l'username e la cifra che vuole ricaricare. A seguito della conferma la cifra viene accreditata. I passaggi da eseguire in questo scenario sono i seguenti:

1. Richiesta dell'username e della cifra da ricaricare;
2. Inserimento dei dati richiesti e generazione del riepilogo della ricarica;
3. Conferma della ricarica.

Il metodo che si occupa di effettuare la ricarica al conto del giocatore è *ricaricaContoGiocatore()* implementato nella classe *WalletController*. Gli scenari di test individuati sono:

- a) Username inserito inesistente. È uno scenario di fallimento;
- b) Importo inserito non valido. È uno scenario di fallimento;
- c) Username esiste e importo valido. È lo scenario di successo.

5.5 MODIFICA SCENARIO DI TEST ISCRIZIONE TORNEO

Nell'iterazione precedente è stato analizzato lo scenario di test di iscrizione al torneo. Nell'iterazione 4 questo scenario ha subito una variazione per quanto riguarda il raggiungimento dello scenario di successo. Ecco le condizioni aggiornate:

- **a)** La richiesta di iscrizione pervenga entro i termini e le date limite stabiliti dal torneo;
- **b)** L'utente che sottoscrive l'iscrizione sia effettivamente presente all'interno della squadra selezionata;
- **c)** Tutti i giocatori che fanno parte della squadra dispongano di un saldo personale sufficiente a coprire la quota d'iscrizione, dove la quota individuale è calcolata come **PrezzoIscrizione / numero**

giocatori in squadra;

- **d)** La disciplina sportiva della squadra coincida perfettamente con lo sport previsto dal torneo.
- **e)** Nessun componente della squadra risulti già iscritto allo stesso torneo all'interno della rosa di un'altra squadra già iscritta.

La modifica della condizione **c** è stata fatta per due motivi:

1. Nel caso di cancellazione del torneo per numero insufficiente di squadre, verrà effettuato un rimborso a tutti gli utenti della squadra e questo elimina la necessità di salvare l'identificativo del giocatore che ha effettuato l'iscrizione;
2. È più pratico, anzi più corretto che i vari giocatori che compongono la squadra contribuiscano al pagamento della quota di iscrizione.

Inoltre, sempre in questa iterazione i test che nell'iterazione 3 non prevedevano il meccanismo di ricarica conto in caso di saldo insufficiente, perché introdotto in questa iterazione, adesso lo prevederanno e verranno adattati.